Template Method

Template Method · Mine CSV/JSON reports with shared pipeline · Build with Waqa

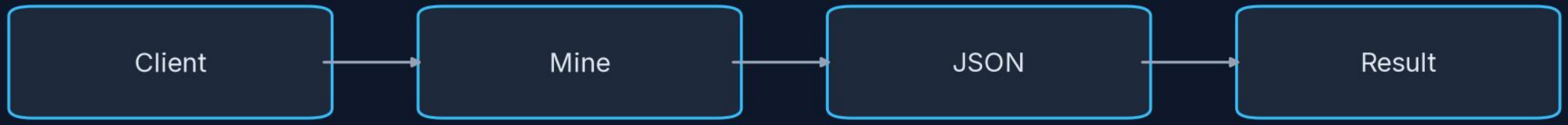
Today's Hook

- Problem: Mine CSV/JSON reports with shared pipeline
- Pattern: Template Method — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Mine CSV/JSON reports with shared pipeline
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Template Method: Mine CSV/JSON reports with shared pipeline

Real-World Scenario

- Running example: Mine CSV/JSON reports with shared pipeline
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.behavioral.templatemethod;

import com.buildwithwaqar.patterns.gof.behavioral.templatemethod.*;
/** Problem: shared ETL skeleton with format-specific parse steps. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Template Method problem: CSV mining ===");
    System.out.println(new CsvMiner().mine("report.csv"));
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.gof.behavioral.templatemethod;

import java.util.List;

public abstract class DataMiner {
    /** Template method — fixed algorithm skeleton. */
    public final String mine(String path) {
        var raw = open(path);
        var data = parse(raw);
        var report = analyze(data);
        close();
        return report;
    }

    protected abstract String open(String path);
    protected abstract List<String> parse(String raw);
    protected String analyze(List<String> data) { return "rows=" + data.size(); }
    protected void close() {}
}

final class CsvMiner extends DataMiner {
    protected String open(String path) { return "a,b\nc,d"; }
    protected List<String> parse(String raw) { return List.of(raw.split("\\n")); }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.gof.behavioral.templatemethod;

import com.buildwithwaqar.patterns.gof.behavioral.templatemethod.*;
/** Problem: shared ETL skeleton with format-specific parse steps. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Template Method problem: CSV mining ===");
    System.out.println(new CsvMiner().mine("report.csv"));
  }
}
```

Walkthrough Output

```
=== Template Method problem: CSV mining ===
rows=2
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Template Method.

Related Patterns

- Contrast Template Method with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Template Method → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.